Assignment No: 8

Name: Atharv Shinde
Class: T.Y IT-A
Batch: D
Roll No: 33076
Subject: DSML

Code:

```
import pandas as pd
from collections import defaultdict
from itertools import combinations

# Load & Preprocess
df = pd.read_csv('Groceries_dataset.csv')
transactions = df.groupby(['Member_number', 'Date'])['itemDescription'].apply(lambda x:
sorted(list(set(x)))).tolist()
N = len(transactions)

# Step 1: Frequent 1-itemsets
min_support = 0.001
min_count = min_support * N

counts = defaultdict(int)
for t in transactions:
    for item in t:
        counts[frozenset([item])] += 1

frequent_itemsets = {itemset: count for itemset, count in counts.items() if count >= min_count}
all_frequent = dict(frequent_itemsets)

# Step 2: Apriori Loop for k >= 2
k = 2
current = frequent_itemsets
while current:
    # Candidate generation
    candidates = set()
    items_list = list(current.keys())
    for i in range(len(items_list)):
        for j in range(i + 1, len(items_list)):
            union_set = items_list[i] | items_list[j]
            if len(union_set) == k:
```

```
                candidates.add(union_set)

    # Candidate counting
    cand_counts = defaultdict(int)
    for t in transactions:
        t_set = set(t)
        for cand in candidates:
            if cand.issubset(t_set):
                cand_counts[cand] += 1

    current = {cand: count for cand, count in cand_counts.items() if count >= min_count}
    all_frequent.update(current)
    k += 1

# Step 3: Rule Generation
rules = []
for itemset, count in all_frequent.items():
    if len(itemset) > 1:
        sup_itemset = count / N
        items = list(itemset)
        for i in range(1, len(items)):
            for ant in combinations(items, i):
                ant = frozenset(ant)
                cons = itemset - ant
                sup_ant = all_frequent[ant] / N
                conf = sup_itemset / sup_ant
                sup_cons = all_frequent[cons] / N
                lift = conf / sup_cons
                if conf >= 0.05:
                    rules.append({'antecedents': list(ant), 'consequents': list(cons), 'support':
sup_itemset, 'confidence': conf, 'lift': lift})

rules_df = pd.DataFrame(rules).sort_values(by='lift', ascending=False)
print(rules_df.head(10).to_string(index=False))
```

O/p:

antecedents	consequents	support	confidence	lift
[whole milk, yogurt]	[sausage]	0.001470	0.131737	2.182917
[sausage, whole milk]	[yogurt]	0.001470	0.164179	1.911760
[specialty chocolate]	[citrus fruit]	0.001403	0.087866	1.653762
[sausage, yogurt]	[whole milk]	0.001470	0.255814	1.619866
[flour]	[tropical fruit]	0.001069	0.109589	1.617141
[beverages]	[sausage]	0.001537	0.092742	1.536764
[whole milk, soda]	[sausage]	0.001069	0.091954	1.523708
[napkins]	[pastry]	0.001738	0.078550	1.518529
[processed cheese]	[root vegetables]	0.001069	0.105263	1.513019
[hard cheese]	[pip fruit]	0.001069	0.072727	1.482586